

Учебный проект на Go

Первая итерация: пользователи и их балансы

Цель итерации

Создать основу будущего маркетплейса, разместить проект на GitHub и закрепить работу со строками, слайсами и мапами.

Начинаем делать учебный проект - небольшой маркетплейс с пользователями, кошельками, товарами и заказами. Будем развивать его постепенно по мере изучения новых тем. На первой итерации достаточно создать репозиторий и реализовать работу с пользователями и их балансами.

1. Создание репозитория

- 1 Если у вас еще нет аккаунта на GitHub, создайте его.
- 2 Создайте публичный репозиторий go-marketplace.
- 3 Добавьте файл README.md.
- 4 Добавьте стандартный .gitignore для Go.
- 5 Склонируйте репозиторий локально.
- 6 Инициализируйте Go-проект командой ниже. Вместо USERNAME укажите свой логин на GitHub.
- 7 Создайте файл main.go.
- 8 Пароли, токены, ключи доступа и настоящие персональные данные в репозиторий добавлять не нужно.

```
go mod init github.com/USERNAME/go-marketplace
```

В README.md пока можно написать:

```
# Go Marketplace

Учебный backend-проект на Go.

Проект будет постепенно расширяться по мере изучения языка.
```

2. Пользователи

Создайте мапу для хранения пользователей:

```
users := map[int]string{}
```

- ключ - уникальный идентификатор пользователя;
- значение - имя пользователя.

2. Пользователи, продолжение

Реализуйте следующие функции:

```
func AddUser(users map[int]string, id int, name string) bool
func GetUser(users map[int]string, id int) (string, bool)
func RenameUser(users map[int]string, id int, newName string) bool
func DeleteUser(users map[int]string, id int) bool
func FindUsersByName(users map[int]string, query string) []int
```

Что должно работать:

- Нельзя добавить пользователя с уже существующим идентификатором.
- Идентификатор должен быть больше нуля.
- Имя не должно быть пустым.
- Переименовать можно только существующего пользователя.
- Искать пользователей нужно по части имени.
- Поиск не должен учитывать регистр.
- Функция поиска возвращает слайс идентификаторов найденных пользователей.

3. Балансы пользователей

Создайте еще одну мапу:

```
balances := map[int]int{}
```

- ключ - идентификатор пользователя;
- значение - баланс пользователя.

Когда добавляется новый пользователь, его баланс должен быть равен нулю.

Реализуйте функции:

```
func TopUpBalance(
    users map[int]string,
    balances map[int]int,
    userID int,
    amount int,
) bool

func GetBalance(
    users map[int]string,
    balances map[int]int,
    userID int,
) (int, bool)
```

3. Балансы пользователей, продолжение

Что нужно учесть:

- Нельзя пополнить баланс несуществующего пользователя.
- Сумма пополнения должна быть больше нуля.
- Баланс не может быть отрицательным.
- Деньги храним целым числом в копейках.

Например:

```
15050 // 150 рублей 50 копеек
```

При удалении пользователя нужно удалить его и из `users`, и из `balances`.

4. Проверка работы

В функции `main` попробуйте:

- 1 Создать пустые мапы `users` и `balances`.
- 2 Добавить трех пользователей.
- 3 Попробовать добавить пользователя с повторяющимся идентификатором.
- 4 Найти пользователя по идентификатору.
- 5 Переименовать одного пользователя.
- 6 Найти пользователей по части имени.
- 7 Пополнить баланс двух пользователей.
- 8 Вывести их балансы.
- 9 Удалить одного пользователя.
- 10 Проверить, что он удалился из обеих мап.

5. Коммиты

Желательно разделять функциональность на отдельные коммиты, чтобы по истории было видно, как постепенно развивался проект.

Например:

```
init project
add user storage and operations
add user balances
add user search
```

Названия могут быть другими. Главное, чтобы один коммит содержал одно понятное изменение.